

BDT training and application

07/03/2026

1. Overview

Samples Used

Sample	Description	Source
Data	2018 UL SingleMuon	Real collision data
DY	Drell-Yan Z/γ^* to tau-tau	MC simulation
ttbar	tt to tau-tau + X	MC simulation
QCD	Multi-jet background	Data-driven (same-sign)

2. Processing Pipeline

NanoAOD --> Phase0 --> Phase1 --> Merge --> Proton Mixing --> Plots

Scripts

Stage	Data	DY	ttbar	QCD
Phase0	fase0_data_mutau.py	fase0_dy.py	fase0.py	-
Phase1	fase1_data.py	fase1_dy.py	fase1_ttjets.py	fase1_qcd.py

3. Phase0: initial skimming

Scripts

- `fase0_data_mutau.py` (Data)
- `fase0_dy.py` (Drell-Yan)
- `fase0.py` (ttbar)

Structure of scripts

- 1 **Trigger selection:** HLT_IsoMu24 (single muon trigger)
- 2 **Luminosity filtering:** Only certified good runs (`dadosluminosidade.txt`)
- 3 **Object selection:** At least 1 muon and 1 tau
- 4 **Variable extraction:** Muon, Tau, Jets, MET kinematics

4. Phase1: Selection cuts

Variable	Cut	Description
muon_id	> 3	Medium MVA ID
tau_id1 (VSjet)	> 63	VeryTight DeepTau vs jets
tau_id2 (VSe)	> 7	Tight DeepTau vs electrons
tau_id3 (VSmu)	> 1	Loose DeepTau vs muons
muon_pt	$> 35 \text{ GeV}$	muon transverse momentum
tau_pt	$> 100 \text{ GeV}$	tau transverse momentum
charge product	< 0	opposite sign
Delta R	> 0.4	Angular separation

Note: QCD uses **same-sign** (SS) selection

5. Event weights in Phase1

Data

```
event_weight = 1.0  # Real data no correction needed
```

Drell-Yan (DY)

```
event_weight = generator_weight x mu_trig_sf x mu_idiso_sf x mu_reco_sf
```

ttbar

```
event_weight = 0.15 x mu_trig_sf x mu_idiso_sf x mu_reco_sf
```

Important: 0.15 already included - should be fixed later for consistency

QCD

```
event_weight = 1.0  # data driven
```

6. Where event weights are applied

Data (no proton mixing needed)

Phase0: $\text{weight}=1.0 \rightarrow$ Phase1: $\text{weight}=1.0 \rightarrow$ Plotting: 1.0

Drell-Yan

Phase1: $\text{event_weight} = \text{generator_weight} \times \text{muon_SFs}$

Proton Mixing: $\text{weight} = \text{event_weight} \times 0.13$

Plotting: $\text{final_weight} = \text{weight} \times 1.81$

7. Where event weights are applied (cont)

ttbar

Phase1: $\text{event_weight} = 0.15 \times \text{muon_SFs}$ <- 0.15 included here

Proton Mixing: $\text{weight} = \text{event_weight} \times 0.13$

Plotting: $\text{final_weight} = \text{weight} \times 1.0$

QCD (data-driven)

Phase1: $\text{weight}=1.0$ -> Plotting: $\text{final_weight}=1.0$

8. Cross-Section normalization factors

Formula used

$\text{weight} = N_{\text{MC}} / N_{\text{expected}}$
where $N_{\text{expected}} = \text{Luminosity} \times \text{Cross-section}$

Values used

Sample	Factor	Included Where?
DY	1.81	Applied in plotting
ttbar	0.15	Already in Phase 1 event_weight

This does not change the outcome but should be fixed for consistency

9. Files

Processing scripts

Script	Purpose
fase0_data_mutau.py	Phase0 for Data
fase0_dy.py	Phase0 for DY
fase1_data.py	Phase1 for Data
fase1_dy.py	Phase1 for DY
fase1_ttjets.py	Phase1 for ttbar
merge_pp_mutau.py	Proton mixing
plot_m.py	Plotting

10. Proton acceptance measurement code

Method in ROOT:

```
TFile* f = TFile::Open("Data_merged_proton_vars.root");
TTree* tree = (TTree*)f->Get("tree");
Long64_t total = tree->GetEntries();
Long64_t good = tree->GetEntries(
    "Sum$(proton_multi_arm==0)>=1 && Sum$(proton_multi_arm==1)>=1"
);
double P = (double)good / (double)total;
cout << "Proton acceptance P = " << P << endl;
```

Result:

```
total events: 6,970
good events: 1,708
P = 0.245 (24.5%)
```

11. Single Lepton Plots: Cuts Applied

Proton requirements

Sample	Cut
Data/QCD	<code>Sum\$(proton_multi_arm==0)>0 && Sum\$(proton_multi_arm==1)>0</code>
MC (DY, ttbar)	<code>xi_arm1_1 >= 0 && xi_arm2_1 >= 0</code>
Signal	<code>xi_arm1_1 >= 0 && xi_arm2_1 >= 0</code>

12. Weights applied

Weights applied

Sample	Weight
Data	None
QCD	None
DY	Proton acceptance (0.245) - xsec normalization already included
ttbar	Proton acceptance (0.245) $\times$ Scale(0.15)
Signal	weight branch (lumi $\times$ xsec $\times$ SFs $\times$ rad. damage) $\times$ 5000 (visualization)

13. Weight fix: old vs new approach

Old weights (MuTauPlots_weight_comparison.C)

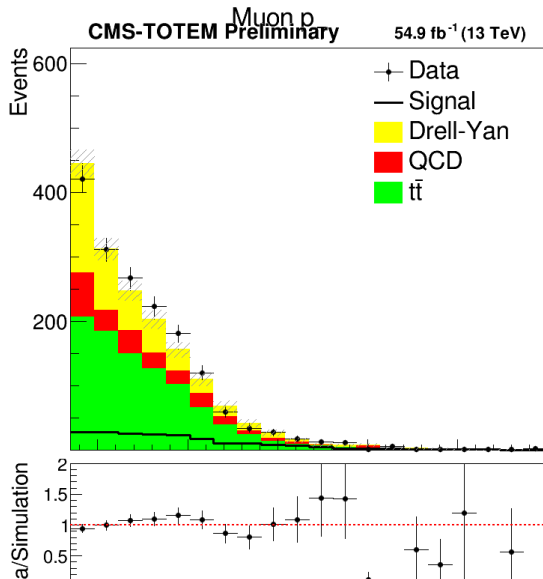
- MC filled with **flat** weight = 0.245 (proton acceptance, no weight branch)
- Then Scale(1.81) for DY, Scale(0.15) for ttbar
- No generator_weight, no muon scale factors

New weights (corrected)

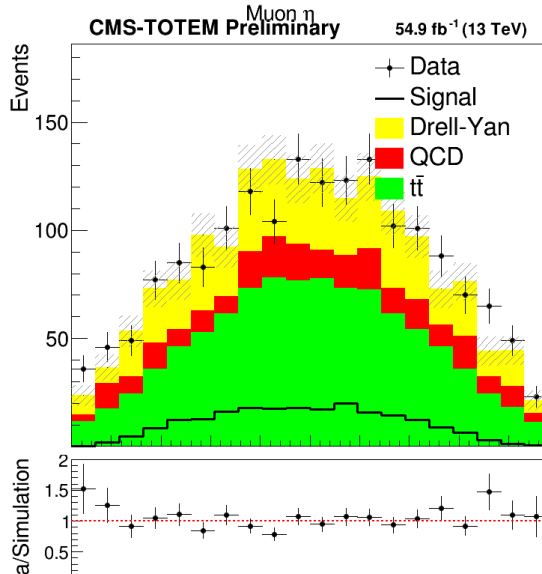
- MC filled using **weight branch** = generator_weight x muon_SF's x 0.245
- Then Scale(1.004e-4) for DY (= $L \times \text{sigma} / \text{Sum_w}$), Scale(1.0) for ttbar

	Old	New
DY weight	0.245 x Scale(1.81)	weight branch x Scale(1.004e-4)
ttbar weight	0.245 x Scale(0.15)	weight branch x Scale(1.0)
DY Scale	1.81 (empirical)	1.004e-4 (= $54900 \times 6077.22 / 3.323e12$)
ttbar Scale	0.15 (empirical)	1.0 (already ~correct via 0.15 in Phase1)

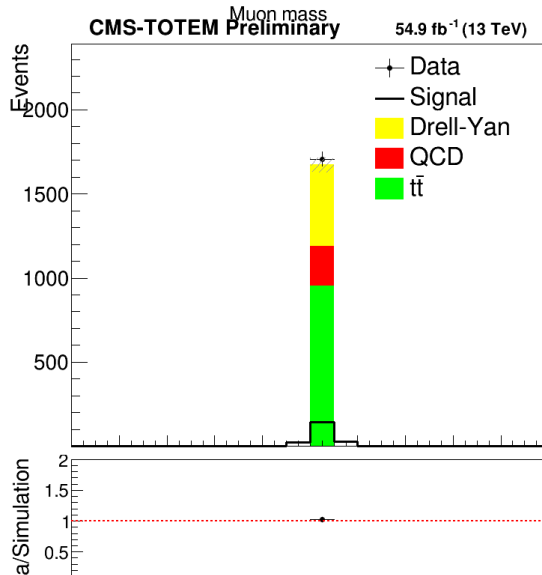
18. Muon pT



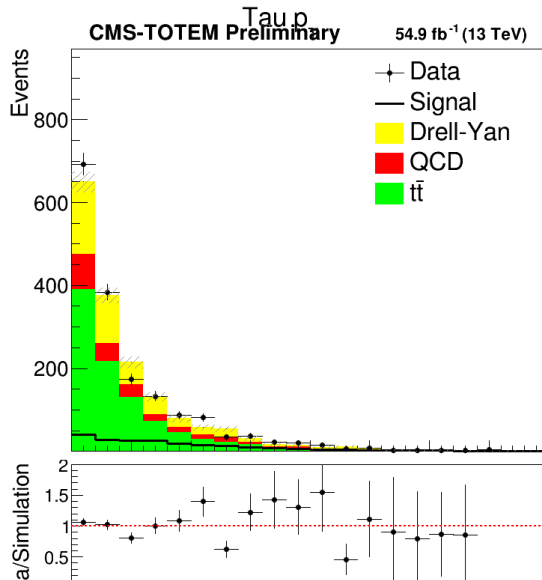
19. Muon eta



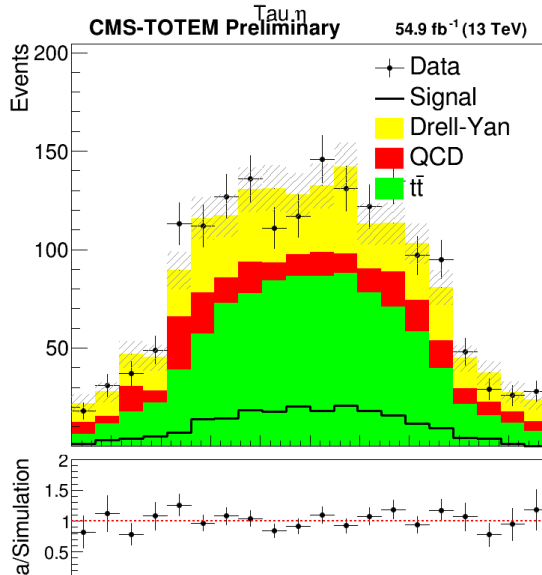
20. Muon mass



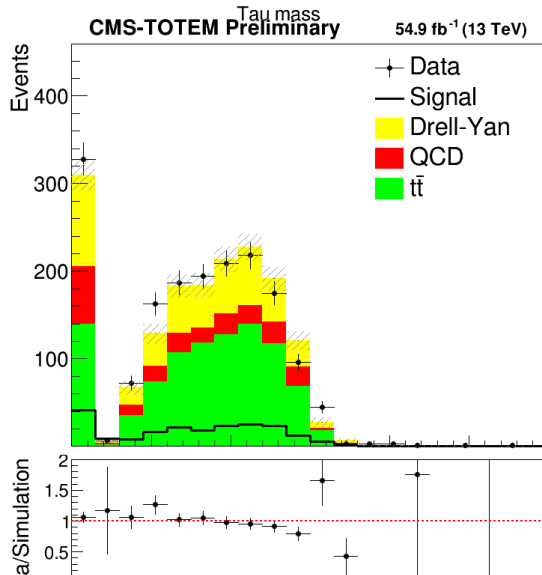
21. Tau pT



22. Tau eta



23. Tau mass



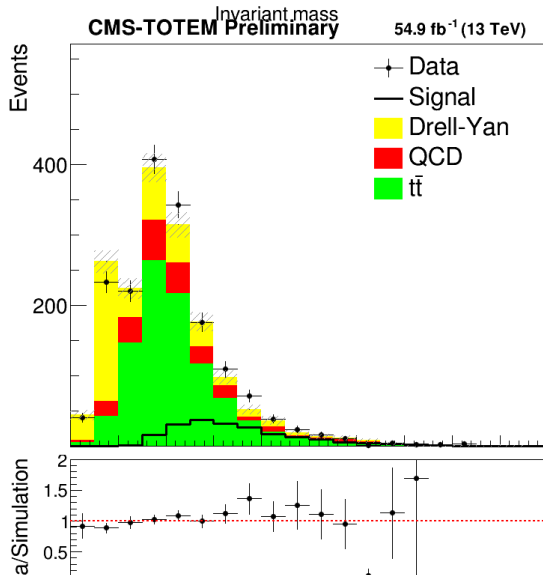
24. Single Distributions After Proton Merging

The following plots show individual kinematic distributions after requiring protons on both PPS arms.

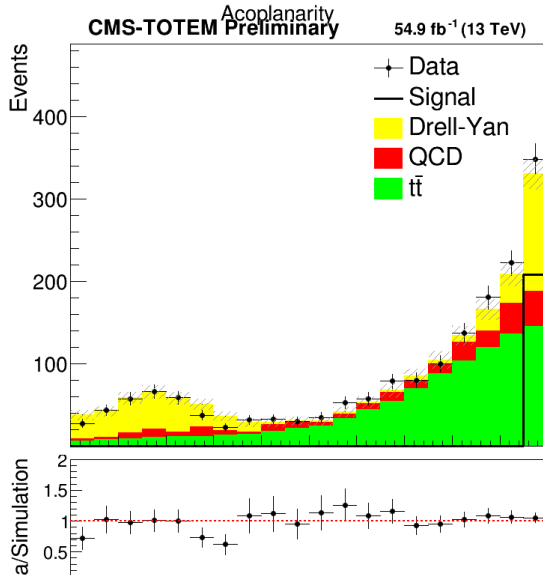
Variables shown

Variable	Description
sist_mass	Invariant mass $m(\mu, \tau)$
acop	Acoplanarity
sist_pt	System transverse momentum
sist_rap	System rapidity
tau_pt	Tau transverse momentum
met_pt	Missing transverse energy

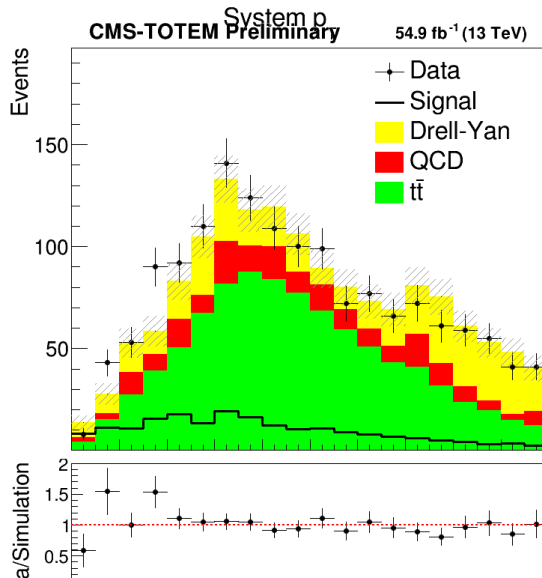
25. Invariant Mass (after proton requirements)



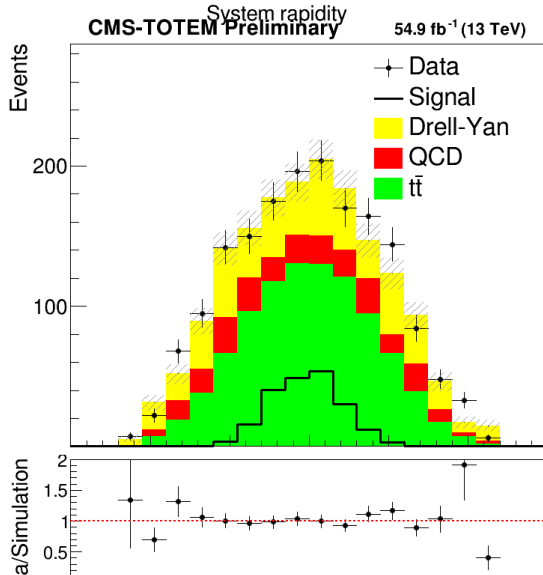
26. Acoplanarity (after proton requirements)



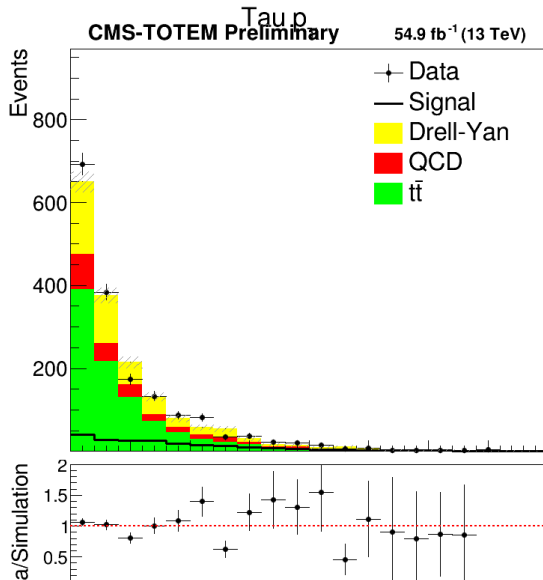
27. System pT (after proton requirements)



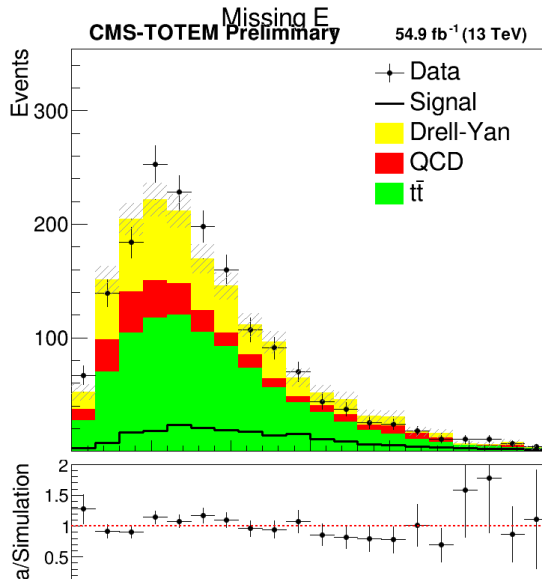
28. System Rapidity (after proton requirements)



29. Tau pT (after proton requirements)



30. MET (after proton requirements)



31. sinal.py: SFs Corrections

Muon Scale Factors Applied

SF Type	Function	Description
Trigger	<code>muon_trig_sf()</code>	IsoMu24 trigger efficiency
ID+Iso	<code>muon_idiso_sf()</code>	Identification and isolation
Reco	<code>mu_reco_sf()</code>	Reconstruction efficiency

Tables loaded from POGCorrections/

- `muon_neg18_lowTrig.txt` - Negative muon, $p_T < 200$ GeV
- `muon_pos18_lowTrig.txt` - Positive muon, $p_T < 200$ GeV
- `muon_18_highTrig.txt` - High p_T muons (> 200 GeV)
- `muon_IdISO.txt` - ID and isolation corrections

32. sinal.py: Event Selection cuts applied

Cut	Requirement
Basic	At least 1 muon and 1 tau
ID	$\text{tau_id_full} > 0.5$ and $\text{mu_id} > 0.5$
Eta	$\text{tau_eta} < 2.4$ and $\text{mu_eta} < 2.4$
Delta R	Angular separation > 0.4
pT	$\text{tau_pt} > 100 \text{ GeV}$, $\text{mu_pt} > 35 \text{ GeV}$
Charge	Opposite sign (product < 0)
Protons	Exactly 2 reconstructed protons

33. BDT summary: workflow

Pipeline

```
corrs_matrix.py --> TMVAClassification.C --> TMVAClassificationApplication.C
(feature QA)      (training)              (evaluation)
```

Files

File	Purpose
corrs_matrix.py	Correlation heatmaps from variables2.txt to select features
TMVAClassification.C	TMVA factory: train BDT + Likelihood + Fisher + MLP + PDERS
TMVAClassificationApplication.C	TMVA reader: apply trained weights, produce MVA score histograms
dataset/weights/*.weights.xml	Serialized trained models

34. BDT summary: training inputs and variables

Input samples

Role	File	Tree
Signal	MuTau_sinal_SM_2018_july.root	tree
Background	background_total-protons_syst.root	tree

Event weights in TMVA

```
dataloader->SetSignalWeightExpression("weight*weights_bsm_sf[51]");  
dataloader->SetBackgroundWeightExpression("weight");  
// Global scale factors:  
double signalWeight = 0.00021;    double backgroundWeight = 104.;
```

9 training variables

Variable	Description
sist_rap	System rapidity
acop	System acoplanarity
sist_pt	System transverse momentum
mu_pt, tau_pt	Lepton transverse momenta
sist_mass	Invariant mass $m(\mu, \tau)$
met_pt	Missing transverse energy
$\text{sist_mass} - \sqrt{13000^2 * \text{xi_arm1_1} * \text{xi_arm2_1}}$	Mass matching (central vs proton)
$\text{sist_rap} - 0.5 * \log(\text{xi_arm1_1} / \text{xi_arm2_1})$	Rapidity matching (central vs proton)

The last two compare the central $\mu + \tau$ system with the forward proton kinematics. For true exclusive events they peak at zero.

BDT results: using simple classifier

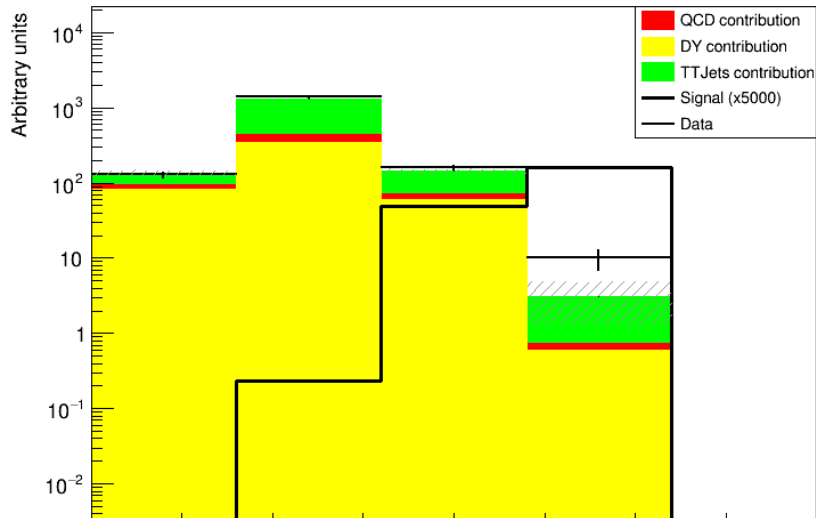
Only BDT with (AdaBoost) is enabled with the default hyperparameters

TMVAClassification → `run_all_bdt.sh` → `bdt_output.cpp`

trains classifier, applies to each sample the results and produces corresponding histograms, plots the histogram of BDT results

BDT results: using simple classifier

Stacked Histogram



35. BDT summary: configuration and output

BDT (AdaBoost) hyperparameters

`NTrees=850, MaxDepth=3, MinNodeSize=2.5%, AdaBoostBeta=0.5,
UseBaggedBoost, BaggedSampleFraction=0.5, GiniIndex, nCuts=20`

Train/test split

- 6500 signal + 15000 background for training, rest for testing
- Random split, `NormMode=NumEvents`
- No additional preselection cuts inside TMVA

36. TMVA methods used

5 classifiers are enabled in `TMVAClassification.C` by default:

Method	Type	Key idea
BDT	Boosted Decision Tree	Ensemble of 850 shallow trees (depth 3), AdaBoost, GiniIndex splitting
Likelihood	Naive Bayes	Per-variable PDF estimation (Spline2 interpolation), assumes uncorrelated inputs
Fisher	Linear discriminant	Optimal linear combination of variables maximizing S/B separation
MLP	Neural network	Multilayer perceptron with tanh activation, hidden layer of N+5 neurons, 600 training cycles
PDERS	Probability density	Adaptive range search in multi-dim space, Gaussian kernel estimator

All five produce per-event MVA scores stored in `TMVApp_sinal.root`.

Application (TMVAClassificationApplication.C)

- Reads trained weights from dataset/weights/
- Computes derived matching variables in the event loop from xi_arm1_1, xi_arm2_1
- Per-event weight branch used as tree weight
- BDT score histogram: 20 bins in $[-0.8, 0.8]$
- Output: TMVApp_sinal.root (contains MVA_BDT, MVA_Likelihood, MVA_Fisher, MVA_MLP, MVA_PDERS)

38. BDT code files: TMVAClassification.C

TMVA factory script that trains the BDT classifier (and optionally other MVA methods). This is the core training step.

Input

- Signal: `MuTau_sinal_SM_2018_july.root` (tree: `tree`)
- Background: `background_total-protons_syst.root` (tree: `tree`)
- ① Registers 9 training variables (kinematic + proton matching)
- ② Sets per-event weight expressions:
 - Signal: `weight * weights_bsm_sf[51]`, global scale = 0.00021
 - Background: `weight`, global scale = 104.0
- ③ Prepares train/test split: 6500 signal + 15000 background for training, random split
- ④ Books the BDT method with AdaBoost (NTrees=850, MaxDepth=3, GinIndex, nCuts=20)
- ⑤ Calls `TrainAllMethods()`, `TestAllMethods()`, `EvaluateAllMethods()`

Output

- `TMVA_allBkg_Mutau_2018.root` - TMVA output with training/testing histograms, ROC curves
- `dataset/weights/TMVAClassification_BDT.weights.xml` - Serialized trained BDT model

39. BDT code files: TMVAClassificationApplication.C

Applies the trained BDT model to the **signal** sample, producing MVA score histograms for evaluation.

Input

- ROOT file: MuTau_sinal_SM_2018_july.root (tree: tree)
- Trained weights: dataset/weights/TMVAClassification_BDT.weights.xml
- ① Creates a TMVA Reader and registers the same 9 variables used in training
- ② Reads branches: acop, sist_pt, sist_mass, sist_rap, met_pt, xi_arm1_1, xi_arm2_1, weight, tau_pt, mu_pt
- ③ For each event, computes the derived matching variables:
 - Mass matching: $\text{sist_mass} - \sqrt{13000^2 * \text{xi1} * \text{xi2}}$
 - Rapidity matching: $\text{sist_rap} - 0.5 * \log(\text{xi1}/\text{xi2})$
- ④ Skips events without valid protons on both arms ($\text{xi} \leq 0$)
- ⑤ Evaluates BDT score via `reader->EvaluateMVA("BDT method")`
- ⑥ Fills BDT score histogram (20 bins, [-0.8, 0.8])

Output

- TMVApp sinal.root - Contains MVA BDT histogram for signal

40. BDT code files: run_bdt_all.C

Standalone BDT application script that can be run on **any** sample type (signal, DY, ttbar, QCD, data). Unlike `TMVAClassificationApplication.C`, this handles the different branch naming and structure across sample types.

Input

- Three arguments: `inputFile`, `outputFile`, `sampleType`
- Sample types: "signal", "dy", "ttbar", "qcd", "data"
- Trained weights: `dataset/weights/TMVAClassification_BDT.weights.xml`
- ① Creates a TMVA Reader with the same 9 training variables
- ② Adapts to sample-specific branch differences:
 - Data/QCD: muon pT branch = `muon_pt`, proton xi from vectors (`proton_multi_arm`, `proton_multi_xi`)
 - MC (signal/DY/ttbar): muon pT branch = `mu_pt`, proton xi from scalars (`xi_arm1_1`, `xi_arm2_1`)
- ③ Loops over events, requires protons on both arms (`xi1 > 0 && xi2 > 0`)
- ④ Computes derived matching variables, evaluates BDT score
- ⑤ Fills weighted histogram `MVA_BDT` (20 bins, [-0.8, 0.8]) using the weight branch

Output

41. BDT code files: run_bdt_all.sh

Input

Sample	Input file
Signal	MuTau_sinal_SM_2018_july.root (local)
DY	/eos/home-m/mblancco/samples_2018_mutau/DY_2018_U
ttbar	/eos/home-m/mblancco/samples_2018_mutau/ttjets_20
QCD	/eos/home-m/mblancco/samples_2018_mutau/QCD_2018_U
Data	/eos/home-m/mblancco/samples_2018_mutau/Data_2018_U

Output

Output file	Content
TMVApp_sinal.root	Signal BDT scores
TMVApp_DY.root	DY BDT scores
TMVApp_ttjets.root	ttbar BDT scores

42. BDT code files: bdt_output.cpp

Plotting script that creates a stacked histogram of BDT output scores, overlaying all backgrounds, signal, and data.

Input

- TMVApp_DY.root - DY BDT scores
- TMVApp_QCD.root - QCD BDT scores
- TMVApp_ttjets.root - ttbar BDT scores
- TMVApp_sinal.root - Signal BDT scores
- TMVApp_data.root - Data BDT scores

What it does

- 1 Opens all five TMVApp_*.root files and extracts the MVA_BDT histogram from each
- 2 Rebins all histograms by factor 4 (20 bins $\rightarrow$ 5 bins)
- 3 Applies normalization: DY Scale($1.004e-4$), Signal Scale(5000) for visibility
- 4 Builds a THStack with colored backgrounds: DY (yellow), QCD (red), ttbar (green)
- 5 Overlays signal (black, line width 3) and data (black points with errors)
- 6 Draws total background uncertainty band (gray hatched)